LAPORAN PRAKTIKUM
ANALISIS KOMPLEKSITAS ALGORITMA

Disusun Oleh:

SAFA AURA

NIM : 2025573010024

KELAS : TI-1D

JURUSAN : Teknologi Informasi dan Komputer

PRODI : Teknik Informatika

DOSEN PENGAJAR : Muhammad Reza Zulman, S.ST., M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan pembahasan mengenai *Analisis Kompleksitas Algoritma* dengan baik.

Materi ini membahas tentang konsep dasar dalam menganalisis efisiensi suatu algoritma, khususnya melalui Big O Notation, Time Complexity, dan Space Complexity. Analisis ini sangat penting dalam dunia pemrograman karena membantu programmer memahami seberapa cepat suatu algoritma berjalan serta seberapa besar penggunaan memori yang dibutuhkan ketika program dijalankan.

Dalam mempelajari materi ini, penulis juga menggunakan beberapa tools pendukung seperti Node.js versi 18 atau lebih baru, Visual Studio Code (VS Code) sebagai editor kode, serta Terminal untuk menjalankan program. Tools tersebut membantu proses penulisan, pengujian, dan analisis algoritma agar dapat dilakukan dengan lebih mudah dan efektif.

Penulis menyadari bahwa pembahasan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar penulisan di masa mendatang dapat menjadi lebih baik.

Akhir kata, penulis berharap materi ini dapat memberikan manfaat serta menambah pemahaman pembaca mengenai analisis kompleksitas algoritma dalam pemrograman.

Buketrata, ... 2026

Safa Aura

2025573010024

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan 2
BAB II 3
LANDASAN TEORI 3
2.1 Pengertian Algoritma 3
2.2 Analisis Kompleksitas Algoritma 3
2.3 Big O Notation 3
2.4 Time Complexity 3
2.5 Space Complexity 4
2.6 Tools yang Digunakan 4
BAB III 5
PEMBAHASAN 5
3.1 Alat dan Bahan 5
3.2 Langkah- Langkah praktikum 5
3.3 Hasil Praktikum 7
3.4 Summit ke Github 9
BAB IV 10
PENUTUPAN 10
4.1 Kesimpulan 10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan pengembangan perangkat lunak, algoritma merupakan bagian yang sangat penting untuk menyelesaikan suatu permasalahan. Algoritma digunakan untuk mengatur langkah-langkah logis agar suatu program dapat berjalan dengan baik dan menghasilkan output yang diinginkan. Namun, tidak semua algoritma memiliki tingkat efisiensi yang sama. Ada algoritma yang dapat menyelesaikan masalah dengan cepat, tetapi ada juga yang membutuhkan waktu lebih lama dan menggunakan memori yang lebih besar.

Oleh karena itu, diperlukan suatu metode untuk menganalisis seberapa efisien suatu algoritma. Salah satu cara yang digunakan adalah analisis kompleksitas algoritma. Analisis ini bertujuan untuk mengetahui kinerja algoritma berdasarkan waktu eksekusi dan penggunaan memori ketika program dijalankan.

Dalam analisis kompleksitas algoritma dikenal konsep Big O Notation, Time Complexity, dan Space Complexity. Big O Notation digunakan untuk menggambarkan tingkat pertumbuhan waktu atau memori yang dibutuhkan oleh suatu algoritma ketika ukuran data semakin besar. Dengan memahami konsep ini, seorang programmer dapat memilih algoritma yang paling efisien untuk digunakan dalam suatu program.

Selain memahami teori, analisis algoritma juga dapat dipraktikkan menggunakan berbagai tools pemrograman seperti Node.js, Visual Studio Code (VS Code), dan Terminal. Dengan menggunakan tools tersebut, proses pembuatan dan pengujian algoritma dapat dilakukan dengan lebih mudah dan sistematis.

Berdasarkan hal tersebut, maka pembahasan mengenai analisis kompleksitas algoritma menjadi penting untuk dipelajari agar dapat memahami bagaimana cara menilai efisiensi suatu algoritma dalam pemrograman.

1.2 Rumusan Masalah

1. Apa yang dimaksud dengan analisis kompleksitas algoritma?
2. Apa yang dimaksud dengan Big O Notation?
3. Apa yang dimaksud dengan Time Complexity?
4. Apa yang dimaksud dengan Space Complexity?
5. Bagaimana penggunaan tools seperti Node.js, VS Code, dan Terminal dalam menganalisis algoritma?

1.3 Tujuan

1. Untuk memahami pengertian analisis kompleksitas algoritma.
2. Untuk mengetahui konsep Big O Notation dalam analisis algoritma.
3. Untuk memahami Time Complexity pada suatu algoritma.
4. Untuk memahami Space Complexity pada suatu algoritma.
5. Untuk mengetahui penggunaan Node.js, VS Code, dan Terminal dalam proses analisis algoritma.

BAB II

LANDASAN TEORI

2.1 Pengertian Algoritma

Algoritma adalah urutan langkah-langkah logis dan sistematis yang digunakan untuk menyelesaikan suatu masalah. Dalam pemrograman, algoritma berfungsi sebagai dasar dalam pembuatan program agar proses penyelesaian masalah dapat dilakukan secara terstruktur dan efisien. Algoritma biasanya dituliskan dalam bentuk pseudocode, flowchart, atau langsung dalam bahasa pemrograman.

2.2 Analisis Kompleksitas Algoritma

Analisis kompleksitas algoritma adalah metode yang digunakan untuk mengukur tingkat efisiensi suatu algoritma. Analisis ini bertujuan untuk mengetahui seberapa besar sumber daya yang dibutuhkan oleh algoritma ketika dijalankan. Sumber daya yang dimaksud biasanya berupa waktu proses dan penggunaan memori. Dengan melakukan analisis kompleksitas, programmer dapat menentukan algoritma mana yang lebih efektif untuk digunakan dalam menyelesaikan suatu permasalahan.

2.3 Big O Notation

Big O Notation adalah cara untuk menyatakan tingkat pertumbuhan kompleksitas suatu algoritma. Notasi ini digunakan untuk menggambarkan seberapa cepat waktu eksekusi atau penggunaan memori meningkat ketika jumlah data yang diproses semakin besar. Big O biasanya ditulis dengan simbol **O()**. Beberapa contoh kompleksitas yang sering digunakan antara lain:

- **O(1)** → Kompleksitas konstan, waktu eksekusi tidak dipengaruhi oleh jumlah data.
- **O(n)** → Kompleksitas linear, waktu eksekusi bertambah sebanding dengan jumlah data.
- **O(n^2)** → Kompleksitas kuadrat, waktu eksekusi meningkat sangat cepat ketika jumlah data bertambah.

2.4 Time Complexity

Time Complexity adalah ukuran yang digunakan untuk mengetahui berapa lama waktu yang dibutuhkan oleh suatu algoritma untuk menyelesaikan suatu proses. Time Complexity tidak dihitung berdasarkan waktu sebenarnya (detik atau menit), tetapi berdasarkan jumlah langkah atau operasi yang dilakukan oleh algoritma. Dengan mengetahui Time Complexity, kita dapat memperkirakan kinerja algoritma ketika memproses data dalam jumlah besar.

2.5 Space Complexity

Space Complexity adalah ukuran yang digunakan untuk mengetahui jumlah memori yang dibutuhkan oleh suatu algoritma 4etika dijalankan. Semakin banyak data yang diproses, biasanya semakin besar pula memori yang dibutuhkan. Oleh karena itu, Space Complexity membantu programmer memahami seberapa efisien penggunaan memori pada suatu algoritma.

2.6 Tools yang Digunakan

1. **Node.js**
   Node.js merupakan lingkungan runtime JavaScript yang digunakan untuk menjalankan kode JavaScript di luar browser. Node.js sering digunakan untuk menguji algoritma dan menjalankan program berbasis JavaScript.

2. **Visual Studio Code (VS Code)**
   Visual Studio Code adalah editor kode yang banyak digunakan oleh programmer. VS Code memiliki berbagai fitur seperti syntax highlighting, debugging, dan extension yang membantu proses penulisan kode menjadi lebih mudah.

3. **Terminal**
   Terminal adalah antarmuka berbasis teks yang digunakan untuk menjalankan perintah-perintah pada sistem operasi. Dalam pemrograman, terminal sering digunakan untuk menjalankan program, mengelola file, serta menguji kode yang telah dibuat.

BAB III

PEMBAHASAN

3.1 Alat dan Bahan

Alat

1. Komputer atau laptop
2. Mouse dan keyboard
3. Akses internet

Bahan

1. Git Bash
2. Visual Studio Code

3.2 Langkah- Langkah praktikum

1. Cd /d

2. Cd "/d/nama folder

3. Cd nama_repository

4. Mkdir praktikum-5

5. Cd praktikum-5

```
HP@LAPTOP-08FUNPL7 MINGW64 ~
$ cd PraktikumStrukturDatadanAlgoritma

HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStruktu
rDatadanAlgoritma
$ cd 225573010024_strukturdataalgoritma

HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStruktu
rDatadanAlgoritma/225573010024_strukturdataal
goritma (main)
$ mkdir praktikum-5

HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStruktu
rDatadanAlgoritma/225573010024_strukturdataal
goritma (main)
$ cd praktikum-5
```

6. Npm init -y

```
HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStruktu
rDatadanAlgoritma/225573010024_strukturdataal
goritma/praktikum-5 (main)
$ npm init -y
Wrote to C:\Users\HP\PraktikumStrukturDatadan
Algoritma\225573010024_strukturdataalgoritma\
praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\
" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

7. Kemudian buka VS Code

8. Ketik : touch 01-intro-kompleksitas.js pada Git Bash

9. Masukkan coding ke aplikasi VS Vode

10. Lalu ketik node 01-intro-kompleksitas.js pada Git Bash

```
HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStruktu
rDatadanAlgoritma/225573010024_strukturdataal
goritma/praktikum-5 (main)
$ touch 01-intri-kompleksitas.js

HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlg
oritma/225573010024_strukturdataalgoritma/praktikum-5 (m
ain)
$ touch 01-intro-kompleksitas.js

HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlg
oritma/225573010024_strukturdataalgoritma/praktikum-5 (m
ain)
$ node 01-intro-kompleksitas.js
=== Perbandingan Waktu Eksekusi ===
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear : 18 ms
O(n^2) cariPasangan : 123 ms

Hasil sama? true
```

11. Lanjutkan ke bagian berikutnya, ulangi seperti Langkah-langkas diatas

12. Ketik touch 02-descructuring-spread.js pada Git Bash.

13. Masukkan coding dan tampilkan outputnya di aplikasi VS Vode.

14. Lalu ketik node 02-descructuring-spread.js pada Git Bash

```
HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlg
oritma/225573010024_strukturdataalgoritma/praktikum-5 (m
ain)
$ touch 02-destructuring-spread.js

HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlg
oritma/225573010024_strukturdataalgoritma/praktikum-5 (m
ain)
$ node 02-destructuring-spread.js
=== O(1) - selalu cepat ===
10
true

=== O(log n) - Binary Search ===
Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) - Linear Search ===
Max dari 1000 elemen: 998.3596740917101

=== O(n^2) - Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2^n) - Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2^n):
2742 ms

Waktu fib(35) O(n) memoization:
0 ms
```

15. Lanjutkan ke bagian berikutnya, ulangi seperti Langkah-langkas diatas

16. Ketik touch 03-space-complexity.js pada Git Bash.

17. Masukkan coding dan tampilkan outputnya di aplikasi VS Vode.

18. Lalu ketik node 03-space-complexity.js pada Git Bash.

```
HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlg
oritma/225573010024_strukturdataalgoritma/praktikum-5 (m
ain)
$ touch 03-space-complexity.js

HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlg
oritma/225573010024_strukturdataalgoritma/praktikum-5 (m
ain)
$ node 03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : undefined
Elemen unik: 7
```

19. Lanjutkan ke bagian berikutnya, ulangi seperti Langkah-langkas diatas

20. Ketik touch 04-refactor.js pada Git Bash.

21. Masukkan coding dan tampilkan outputnya di aplikasi VS Vode.

22. Lalu ketik node 04-refactor pada Git Bash.

```
HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlg
oritma/225573010024_strukturdataalgoritma/praktikum-5 (m
ain)
$ touch 04-refactor.js

HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlg
oritma/225573010024_strukturdataalgoritma/praktikum-5 (m
ain)
$ node 04-refactor.js
Duplikat LAMBAT O(n^2): 1 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 2, b: 2, c: 1, d: 1 }
```

23. Kemudian kerjakan latihan-1 dan Latihan-3, tapi karna yang dibutuhkan 1000, maka output yang berjalan membutuhkan waktu sangat lama.

3.3 Hasil Praktikum

1. Ketik mkdir tugas, lalu touch tugas/tugas-1.js tugas/tugas-2.js seperti dibawah

```
HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlgoritma/225573010024_stru
kturdataalgoritma/praktikum-5 (main)
$ mkdir tugas

HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlgoritma/225573010024_stru
kturdataalgoritma/praktikum-5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

2.output tugas-1.js

```
C:\Program Files\nodejs\node.exe .\tugas\tugas-1.js
=== INTERSECTION ===
> Lambat: (2) [2, 3]
> Cepat : (2) [2, 3]

=== ANAGRAM ===
[eat, tea, ate] [tan, nat] [bat]

=== KUADRAT ===
Lambat: true
Cepat : false

=== BENCHMARK ===
Intersection Lambat: 540 ms
Intersection Cepat : 6 ms
Kuadrat Lambat: 6 ms
Kuadrat Cepat : 5 ms
```

3.output tugas-2.js

```
C:\Program Files\nodejs\node.exe .\tugas\tugas-2.js

=== n = 100 ===
O(1)    : 1 ms
O(n)    : 0 ms
O(nlogn): 0 ms
O(n^2)  : 4 ms

=== n = 500 ===
O(1)    : 0 ms
O(n)    : 0 ms
O(nlogn): 0 ms
O(n^2)  : 31 ms
```

```
=== n = 1000 ===
O(1)    : 0 ms
O(n)    : 0 ms
O(nlogn): 0 ms
O(n^2)  : 6 ms

=== n = 5000 ===
O(1)    : 0 ms
O(n)    : 0 ms
O(nlogn): 0 ms
O(n^2)  : 1268 ms

=== n = 10000 ===
O(1)    : 0 ms
O(n)    : 0 ms
O(nlogn): 1 ms
O(n^2)  : 630 ms
```

3.4 Summit ke Github

1. masuk git bash, lalu ketik :

2. Cd /d

3. Masuk ke folder yang sudah disiapkan sebelumnya Cd "/d/praktikum struktur data & algoritma"

4. Masuk ke repository : cd 225573010024_strukturdataalgoritma

5. Kemudian masuk ke folder praktikum-5: cd praktikum-5

6. Kembali ke repository : cd ..

7. Ketik : git status lalu git add praktikum-5/

```
HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlgoritma/225573010024_stru
kturdataalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   praktikum-2/01-variabel.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        05-stack-queue.js
        praktikum-5/

no changes added to commit (use "git add" and/or "git commit -a")

HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlgoritma/225573010024_stru
kturdataalgoritma (main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced
by CRLF the next time Git touches it
```

8. Ketik : git commit -m "praktikum 5: Analisis Kompleksitas Algoritma"

9. Ketik :git push origin main.

```
HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlgoritma/225573010024_stru
kturdataalgoritma (main)
$ git commit -m "Praktikum 5: Analisis Komplesitas Algoritma"
[main b995d25] Praktikum 5: Analisis Komplesitas Algoritma
 9 files changed, 420 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-destructuring-spread.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/latihan-3.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js

HP@LAPTOP-08FUNPL7 MINGW64 ~/PraktikumStrukturDatadanAlgoritma/225573010024_stru
kturdataalgoritma (main)
$ git push origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 2 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (13/13), 5.01 KiB | 855.00 KiB/s, done.
Total 13 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/safaaura27/225573010024_strukturdataalgoritma.git
   d561e24..b995d25  main -> main
```

10. buka github di browser, dan buka bagian repository, selesai.

BAB IV

PENUTUPAN

4.1 Kesimpulan

Berdasarkan pembahasan yang telah dilakukan mengenai analisis kompleksitas algoritma, dapat disimpulkan bahwa algoritma merupakan langkah-langkah logis dan sistematis yang digunakan untuk menyelesaikan suatu permasalahan dalam pemrograman. Algoritma memiliki peranan penting dalam menentukan bagaimana sebuah program dapat bekerja secara efektif dan menghasilkan output yang sesuai dengan tujuan yang diharapkan.

Analisis kompleksitas algoritma digunakan untuk mengetahui tingkat efisiensi suatu algoritma, terutama dalam hal waktu eksekusi dan penggunaan memori. Dengan melakukan analisis tersebut, seorang programmer dapat membandingkan beberapa algoritma dan memilih algoritma yang paling efisien untuk digunakan dalam suatu program.

Konsep Big O Notation digunakan untuk menggambarkan tingkat pertumbuhan kompleksitas suatu algoritma ketika jumlah data yang diproses semakin besar. Melalui Big O Notation, kita dapat memahami bagaimana kinerja algoritma berubah ketika ukuran data meningkat. Selain itu, terdapat dua aspek penting dalam analisis algoritma yaitu Time Complexity yang mengukur waktu yang dibutuhkan oleh algoritma untuk menjalankan suatu proses, serta Space Complexity yang mengukur jumlah memori yang digunakan oleh algoritma saat dijalankan.

Dalam praktiknya, proses analisis algoritma dapat dilakukan dengan menggunakan berbagai tools seperti Node.js untuk menjalankan program JavaScript, Visual Studio Code sebagai editor kode, serta Terminal untuk mengeksekusi perintah dan menjalankan program. Dengan menggunakan tools tersebut, proses pengujian dan analisis algoritma dapat dilakukan secara lebih mudah dan terstruktur.

Dengan memahami konsep algoritma, analisis kompleksitas, serta penggunaan tools pemrograman, diharapkan mahasiswa dapat lebih memahami cara menilai efisiensi suatu algoritma serta mampu menerapkannya dalam pembuatan program yang lebih optimal.